

中山大学

《软件工程》期末大作业

逸仙活动云

软件工程化（自动化、协作化）说明文档

项目名称： 校园活动信息平台（逸仙活动云）

组长学号： _____

组长姓名： _____

课程教师： _____

提交日期： 2026 年 _____ 月 _____ 日

版本： 1.0 文档状态： 提交版

目录

1	工程化目标	1
2	已采用的工程实践	2
3	协作流程	3
4	自动化流水线	3
5	质量门禁与完成定义	4
6	当前缺口与改进计划	4
7	工程资产核对表	5
8	协作产物的最低要求	5
9	过程度量与改进闭环	5

文档范围：本报告依据前端仓库（Vue 3）与后端仓库（Flask）截至 2026-07-18 的代码、配置及测试资产编写。**术语说明：**“已实现”指代码库已有对应实现；“计划”指建议在交付/运维阶段落实，二者不混同。

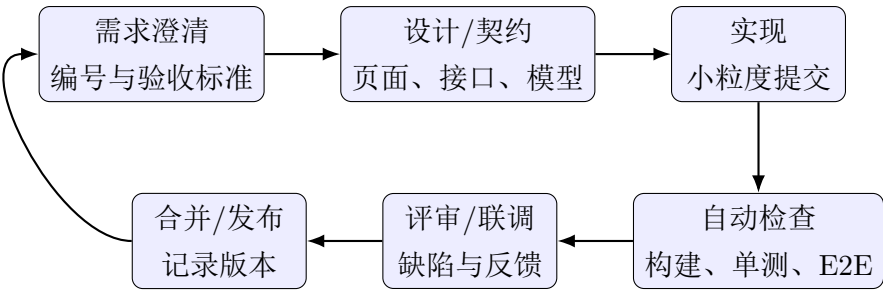
1 工程化目标

本项目以“每次变更可追踪、可构建、可验证、可交接”为工程化目标，围绕软件过程、协作、自动化和知识沉淀建立轻量但可执行的机制。其核心不是堆砌工具，而是让需求—设计—代码—测试—发布形成闭环，并使前后端能在接口演化中保持一致。

2 已采用的工程实践

方向	已有实践与证据	作用
过程与文档	前端采用“先设计、后开发、再联调与归档”流程； <code>docs/frontend-design</code> 记录页面设计， <code>frontend-plan</code> 记录工作清单和契约边界。	让页面结构、接口、Mock 和验收标准可审阅，减少口头约定。
模块化实现	Vue 按 <code>views</code> 、 <code>components</code> 、 <code>api</code> 、 <code>stores</code> 、 <code>composables</code> 分层；Flask 按应用工厂、蓝图、服务、模型分层。	降低耦合，使业务规则、界面和基础设施可独立修改、测试。
自动构建	前端 <code>npm run build</code> 执行 <code>vue-tsc</code> 类型检查和 Vite 生产构建；后端使用 <code>requirements</code> 锁定运行依赖入口。	及早发现类型/编译问题并生成可部署静态资源。
自动测试	前端配置 Vitest 与 Playwright， <code>npm run verify</code> 串行执行构建、单元和端到端测试；后端 <code>pytest</code> 覆盖认证、活动、审计、数据源、搜索、质量与安全服务。	将关键回归从人工检查转为可重复执行的检查。
持续集成	GitHub Actions 工作流在 <code>push</code> 与 <code>pull request</code> 上执行 <code>npm ci</code> 、Playwright 浏览器安装、 <code>build</code> 、 <code>unit</code> 、 <code>e2e</code> 。	把合并门槛前移，避免本地“能跑”掩盖干净环境失败。
环境一致性	<code>.env.example</code> 、Vite API 配置、 <code>Dockerfile</code> 、 <code>Docker Compose</code> 、 <code>Gunicorn</code> 配置。	统一开发/部署变量与后端服务拓扑，降低“在我机器上可用”风险。

3 协作流程



建议以 Issue/任务卡承载一个可验收工作项，描述需求编号、影响范围、完成定义和风险；每个功能分支只解决一个主题。合并请求需说明变更、验证命令、截图/接口证据和文档更新；至少一名非作者成员检查权限、异常路径、可读性和测试。紧急缺陷可走快速通道，但必须补齐测试与复盘。

4 自动化流水线

阶段	命令/触发	产出与门禁
本地开发	node mock/index.js、 npm run dev	Mock 与前端快速联调；变更前后执行相关单测。
前端验证	npm run build; npm run test:unit; npm run test:e2e; npm run verify	类型检查、生产构建、组件/工具测试及浏览器用户流均通过。
后端验证	pytest（在 back- end 虚拟环境）	以内存 SQLite 和测试配置验证 API、服务、权限、质量与安全逻辑。
CI	push/PR 触发 GitHub Actions	干净 Node 22 环境执行安装、构建、单测和 Playwright E2E；失败不得合并。
部署准备	Docker 构建与 Compose 启动， 健康检查	生成镜像、检查服务依赖，使用 /api/health 验证存活。

5 质量门禁与完成定义

一个任务达到“完成”至少需满足：需求/设计变更已记录；代码通过格式与类型/语法检查；新增或受影响的单元测试通过；涉及跨端流程时 E2E 或手工联调有证据；权限、加载、空态和异常路径被检查；接口契约、配置示例或运维文档按需更新；评审意见已处理。CI 未通过、含真实密钥、未解释的跳过测试或破坏主干构建均禁止合并。

6 当前缺口与改进计划

当前前端 CI 已落地，后端测试资产较完整，但后端独立 CI、静态检查/格式化门禁、覆盖率阈值、依赖漏洞扫描、镜像扫描和部署后冒烟尚应补齐。建议优先级为：

1. 新建后端 CI：安装 requirements-dev，运行 pytest、静态检查和覆盖率报告；
2. 将前后端契约样例纳入契约测试，阻止字段/时区/分页回归；
3. 以受保护主分支、必需评审和必需 CI 状态实施合并策略；
4. 发布流水线增加镜像 tag、健康检查、迁移、回滚和制品留存；
5. 用 Issue 看板统计缺陷、返工和交付周期，在迭代回顾中持续改进。

这些改进被明确列为计划，不应误表述为当前已启用能力。

7 工程资产核对表

资产	仓库证据	审校结论
前端构建与测试	<code>package.json</code> 的 <code>build</code> 、 <code>test:unit</code> 、 <code>test:e2e</code> 、 <code>verify</code> 脚本。	已配置；报告不将脚本存在等同于本次执行成功，结果见测试质量报告。
前端 CI	<code>.github/workflows/ci.yml</code> ，已实现自动化 workflow；仍需确认在代码托管平台触发 <code>push</code> /PR，安装依赖和 Chromium 后执行 <code>build/unit/e2e</code> 。	已实现自动化 workflow；仍需确认在代码托管平台触发 <code>push</code> /PR，安装依赖和 Chromium 后执行 <code>build/unit/e2e</code> 。
前端设计过程	<code>docs/frontend-design</code> 已形成设计—实现—联调的文档资产；变更时 <code>docs/frontend-plan</code> 、开发规范和契约矩阵。	已形成设计—实现—联调的文档资产；变更时应同步更新。
后端测试资产	<code>backend/tests</code> 的 <code>pytest</code> 测试、 <code>Test-Config</code> 、内存 <code>SQLite</code> <code>fixture</code> 。	测试代码存在；当前虚拟环境未安装 <code>pytest</code> ，不能声称已执行通过。
后端部署资产	<code>Dockerfile</code> 、 <code>Compose</code> 、 <code>Gunicorn</code> 、 <code>.env.example</code> 。	可作为可复现部署起点；生产迁移、监控与密钥管理仍为计划。

8 协作产物的最低要求

除团队报告外，每项协作应留下可审计产物：需求卡或 Issue、设计/契约链接、关联分支或提交、评审记录、自动化运行链接、缺陷单和发布说明。前后端接口变更必须由双方确认字段、枚举值、分页、时区、权限和错误语义；仅在聊天中确认而未更新契约矩阵，不得视为完成。这样可将“协作化”从口号转化为可检查证据。

9 过程度量与改进闭环

每个迭代结束统计计划完成率、PR 首次通过率、构建失败次数、自动化通过率、P1/P2 缺陷存量、缺陷修复周期和返工原因。对失败执行五问或因果分析，产出可行动项、负责人和截止日期；下一轮回顾检查行动项是否关闭。度量只用于过程改进，不以数量替代代码与产品质量。